Guía de ejercicio de Archivos

Parte 1: Conceptos Básicos y Tipos Simples

1. Crear una función que permita al usuario ingresar 10 números enteros y los guarde en un archivo llamado numeros.dat.
2. Realizar una función que abra el archivo numeros.dat (creado en el ejercicio anterior), lea todos los números, los muestre por pantalla y calcule su promedio.
3. Diseñar una función que solicite números reales al usuario (hasta que ingrese un 0) y los vaya agregando al final de un archivo reales.dat. *(Nota: Si el archivo ya existe, los datos deben agregarse sin borrar los anteriores)*.

Parte 2: Archivos de Registros (Structs)

Para los siguientes ejercicios, definiremos la siguiente estructura:

```
struct Alumno {
  int legajo;
  char nombre[50];
  float promedio;
  bool activo; // Para borrado lógico
};
```

4. **Alta de Registros:** Crear una función agregarAlumno que permita ingresar los datos de un alumno y lo guarde en el archivo alumnos.dat.
5. **Listado Completo:** Realizar una función listarAlumnos que lea el archivo alumnos.dat registro por registro y muestre los datos de todos los alumnos.
6. **Búsqueda Secuencial:** Diseñar una función que reciba un número de legajo, busque ese alumno en el archivo y, si lo encuentra, muestre su nombre y promedio. Si no existe, informar con un mensaje.
7. **Filtro:** Crear una función que genere un archivo de texto aprobados.txt (legible por humanos) que contenga solamente los nombres y promedios de los alumnos que tengan promedio superior a 6.

Vamos subiendo la complejidad *(Modificación y Borrado)*

8. **Modificación de un campo:** Realizar una función que pida un legajo al usuario. Si el alumno existe, debe permitir modificar su **promedio**. *(Estrategia: Buscar el registro, posicionar el puntero del archivo en la posición correcta usando fseek, y sobrescribir el registro modificado)*.
9. **Baja Lógica:** Diseñar una función que reciba un legajo y realice una "baja lógica". Esto significa buscar el registro y cambiar el campo activo a false, sobrescribiendo el registro en el archivo. No se borra físicamente el dato, solo se marca.
10. **Listado de Activos:** Modificar la función del ejercicio 5 para que solo muestre por pantalla a los alumnos que tengan el campo activo en true.

Vamos a subir la complejidad un poco más *(Integración: Vectores + Archivos y Procesos de Lote)*

11. **De Archivo a Vector:** Dado un archivo de alumnos.dat con, a lo sumo, 100 registros, cargar todos los alumnos activos en un vector en memoria. *(Esto se hace para poder ordenar o trabajar más rápido)*.
12. **Ordenamiento y Back-up:** Tomando el vector cargado en el ejercicio anterior, ordenarlo alfabéticamente por nombre. Luego, guardar este listado ordenado en un nuevo archivo llamado alumnos_ordenados.dat.
13. **Baja Física (Compactación):** Dado el archivo alumnos.dat que contiene registros activos (true) e inactivos (false), generar un archivo temporal que contenga **solo** los activos. Luego, eliminar el archivo original y renombrar el temporal como alumnos.dat.
14. **Corte de Control (Simulado):** Se tiene un archivo ventas.dat con registros (codigo_producto, cantidad_vendida). El archivo está **ordenado** por código de producto. Se pide leer el archivo y mostrar el total de unidades vendidas por cada producto. *(Ejemplo: Si hay 3 registros del producto 'A' con cantidades 10, 5 y 5, mostrar "Producto A: 20 ventas total")*.
15. **Apareo de Archivos (Merge):** Se tienen dos archivos cursoA.dat y cursoB.dat, ambos ordenados por legajo de menor a mayor. Generar un tercer archivo todos.dat que contenga a los alumnos de ambos cursos, manteniendo el orden por legajo. *(Nota: Se debe leer de a un registro de cada archivo y comparar para decidir cuál escribir primero)*.
16. **Actualización de Stock (Maestro-Novedades):**
    - **Maestro:** productos.dat (id, descripcion, stock_actual).
    - **Novedades:** ventas_del_dia.dat (id, cantidad_vendida).
    - **Consigna:** Recorrer el archivo de ventas y, por cada venta, buscar el producto correspondiente en el archivo maestro y restar la cantidad vendida al stock actual. Actualizar el archivo maestro.